E4 Educator v2.0

T02

GPIO and build_flags in depth

Tier 1 · Tutorial 2 of 7

Walark Electronics · Fundrum Engineering · May 2026

In this tutorial

You will learn:

- How ESP32 GPIO pins work — modes, drive strength, and limitations
- The difference between INPUT, INPUT_PULLUP and OUTPUT
- Why certain GPIO pins have special behaviour at boot
- How build_flags turn platformio.ini defines into usable constants in your code
- How to read analog signals correctly using ADC1
- PWM output using the LEDC peripheral — dimming the TFT backlight
- The E4 v2.0 full pin reference and which pins to use for what

You will need:

- E4 Educator v2.0 board with ESP32 fitted
- T01 completed — platformio.ini template ready
- USB-C cable (data-capable)

1 GPIO fundamentals

GPIO stands for General Purpose Input/Output. On the ESP32, most pins can be configured as either inputs or outputs in software — but not all pins are created equal. Understanding the differences saves a lot of debugging time.

1.1 The three GPIO modes

Mode	pinMode() call	Use when...
OUTPUT	`pinMode(pin, OUTPUT)`	Driving an LED, buzzer, transistor base, or any active output
INPUT	`pinMode(pin, INPUT)`	Reading a signal that is always driven — a sensor with its own output driver. Avoid leaving inputs floating.
INPUT_PULLUP	`pinMode(pin, INPUT_PULLUP)`	Reading a button or switch to GND. The internal pull-up holds the pin HIGH until the button pulls it LOW. This is the correct mode for all E4 buttons.

Key point

Always use INPUT_PULLUP for buttons wired to GND — never INPUT. A floating input (INPUT with nothing connected) picks up electrical noise and gives random readings. INPUT_PULLUP gives a clean, stable HIGH until the button is pressed.

1.2 Input-only pins

GPIO 34, 35, 36, and 39 are input-only — they have no output driver. You cannot call digitalWrite() on them and expect anything to happen. They are used on the E4 for:

- GPIO 34 — I2S SD data (MEMS microphone)
- GPIO 35 — I2S WS word select (MEMS microphone)
- GPIO 36 — Analog microphone input (MAX4466)
- GPIO 39 — LDR light sensor input

Despite being input-only at the GPIO level, these pins work correctly with hardware peripherals like I2S and ADC — the peripheral's internal routing bypasses the output restriction.

1.3 Boot-sensitive pins

Several ESP32 GPIO pins are sampled at power-up to determine the boot mode. If they're in an unexpected state, the chip may fail to boot or behave strangely. The E4 v2.0 has been designed to manage these carefully — but you should understand which pins are involved.

GPIO	E4 assignment	Boot behaviour	Notes
`GPIO 0`	BOOT button (front)	Must be HIGH or floating	Internal pull-up holds HIGH. The BOOT button pulls it LOW deliberately to enter flash mode.
`GPIO 5`	TFT_CS	Outputs PWM at boot	TFT_eSPI handles this correctly. The TFT may flicker briefly at power-up — this is normal.

GPIO 12	TFT_BL backlight	Affects flash voltage	Keep LOW or floating at boot. Initialise backlight after setup() begins.
GPIO 15	SD_CS	Must be HIGH at boot	10k pull-up fitted on board ensures correct boot state.

Boot sequence in your firmware

Always initialise GPIO 12 (TFT_BL) early in setup() with digitalWrite(TFT_BL, LOW) before enabling LEDC PWM. This gives the backlight a known safe state before you bring it up intentionally.

2 build_flags in depth

In T01 we used build_flags without fully explaining what happens under the hood. Let's fix that — because understanding this makes you a significantly better embedded developer.

2.1 What -D does

When you write this in platformio.ini:

```
build_flags = -DLED_RED=16
```

The compiler sees exactly the same thing as if you had written this at the very top of every .cpp and .h file in your project:

```
#define LED_RED 16
```

The -D prefix means "define". The compiler substitutes LED_RED with 16 everywhere it appears in your code before compiling. It's a text substitution that happens before a single line of your code is compiled.

Key point
build_flags defines are global — they apply to every source file in the project automatically. You never need to #include anything or pass them as parameters. They are simply always available everywhere.

2.2 String defines

Strings in build_flags need escaped quotes — this is the one syntax quirk worth remembering:

```
; In platformio.ini:
build_flags = -DFW_VERSION=\"1.0.0\"

; In your code this becomes:
#define FW_VERSION "1.0.0"

; Use it like this:
Serial.println(FW_VERSION);   // prints: 1.0.0
Serial.printf("FW: %s\n", FW_VERSION);
```

2.3 Using defines in code

Here is the full pattern — platformio.ini defines flowing cleanly into firmware code:

```
; platformio.ini
build_flags =
  -DLED_RED=16
  -DLED_GREEN=26
  -DLED_BLUE=32
  -DBTN_NEXT=13
  -DBTN_ENT=14
  -DTFT_BL=12
```

```
  -DFW_VERSION=\"1.0.0\"
```

```
// main.cpp — no GPIO numbers anywhere in this file
#include <Arduino.h>

void setup() {
  Serial.begin(115200);
  Serial.printf("Firmware v%s\n", FW_VERSION);

  pinMode(LED_RED,   OUTPUT);
  pinMode(LED_GREEN, OUTPUT);
  pinMode(LED_BLUE,  OUTPUT);
  pinMode(BTN_NEXT,  INPUT_PULLUP);
  pinMode(BTN_ENT,   INPUT_PULLUP);

  // Initialise backlight safely
  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);
}
```

Notice: not a single raw GPIO number anywhere in main.cpp. If the hardware ever changes and LED_RED moves to a different pin, you change one line in platformio.ini and every reference in every file updates automatically.

3 Analog input — ADC1 and the WiFi rule

The ESP32 has two ADC (Analog to Digital Converter) units — ADC1 and ADC2. They look identical in code but behave very differently when WiFi is active.

3.1 The ADC2 problem

ADC2 is shared with the WiFi radio. When WiFi is active, the radio takes control of ADC2 and analog reads from ADC2 pins either fail silently or return garbage values. This has caught out many experienced developers.

⚠ The ADC rule — memorise this

When WiFi is active, only use ADC1 pins for analog reads. ADC1 pins on the ESP32 are GPIO 32, 33, 34, 35, 36, and 39. ADC2 pins include GPIO 0, 2, 4, 12, 13, 14, 15, 25, 26, 27 — avoid these for analog when WiFi is running.

3.2 E4 analog pins

The E4 v2.0 correctly uses ADC1 pins for all analog inputs:

Define	GPIO	ADC	Connected to
LDR	GPIO 39	ADC1 Ch3	LDR light sensor voltage divider
MIC_ANALOG	GPIO 36	ADC1 Ch0	MAX4466 analog microphone output

3.3 Reading an analog value

analogRead() returns a 12-bit value — 0 to 4095 — corresponding to 0V to 3.3V. The mapping is not perfectly linear on the ESP32 but is accurate enough for most sensor applications.

```
void loop() {
  // Read the LDR light sensor
  int rawLDR = analogRead(LDR);

  // Convert to a 0-100 percentage (simple map)
  int lightPct = map(rawLDR, 0, 4095, 0, 100);

  Serial.printf("LDR raw: %d  Light: %d%%\n", rawLDR, lightPct);

  // Read the analog microphone
  int rawMic = analogRead(MIC_ANALOG);
  Serial.printf("Mic raw: %d\n", rawMic);
}
```

★ Key point

The map() function scales a value from one range to another. map(rawLDR, 0, 4095, 0, 100) converts the 12-bit ADC reading to a 0-100 percentage. It's a useful tool for making raw sensor readings meaningful.

4 PWM output — dimming the backlight with LEDC

PWM (Pulse Width Modulation) is how the ESP32 creates variable output levels from a digital pin. Instead of the pin being simply HIGH or LOW, it switches rapidly between the two — and the proportion of time spent HIGH versus LOW controls the effective output level.

The ESP32 has a dedicated PWM peripheral called LEDC (originally designed for LED control, but usable for any PWM purpose). It supports up to 16 independent PWM channels.

4.1 LEDC setup

LEDC requires three setup steps: configure a channel, attach it to a pin, then write a duty cycle value.

```
#include <Arduino.h>

// LEDC channel configuration
const int BL_CHANNEL  = 0;      // LEDC channel 0-15
const int BL_FREQ     = 5000;   // 5kHz PWM frequency
const int BL_RES      = 8;      // 8-bit resolution (0-255)

void setup() {
  Serial.begin(115200);

  // Initialise backlight pin LOW first (boot safety)
  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);

  // Set up LEDC channel
  ledcSetup(BL_CHANNEL, BL_FREQ, BL_RES);

  // Attach the channel to the backlight pin
  ledcAttachPin(TFT_BL, BL_CHANNEL);

  // Start at full brightness
  ledcWrite(BL_CHANNEL, 255);
  Serial.println("Backlight on at full brightness");
}

void loop() {
  // Fade from full brightness to off and back
  for (int brightness = 255; brightness >= 0; brightness -= 5) {
    ledcWrite(BL_CHANNEL, brightness);
    delay(20);
  }
  for (int brightness = 0; brightness <= 255; brightness += 5) {
    ledcWrite(BL_CHANNEL, brightness);
    delay(20);
  }
}
```

Note: this exercise uses delay() inside the for loops — this is acceptable here because the fade itself is the entire purpose of the loop. In a real project you would drive the fade with millis() as covered in T01.

4.2 LEDC parameters explained

Parameter	E4 value	Explanation
Channel	`0`	One of 16 independent LEDC channels (0-15). Each attached pin needs its own channel.
Frequency	`5000 Hz`	PWM switching speed. 5kHz is above audible range (avoids whine) and well within LEDC capability.
Resolution	`8 bits`	Duty cycle precision. 8-bit gives 256 steps (0-255). Higher resolution = smoother dimming.
Duty cycle	`0-255`	0 = fully off, 255 = fully on. 128 = 50% duty = roughly half brightness.

5 E4 v2.0 full pin reference

This is your complete reference for every pin on the E4 Educator v2.0. Keep it handy — you'll refer to it throughout the course.

Define	GPIO	Function	Mode	Notes
`LED_RED`	16	Red LED	OUTPUT	330Ω series resistor fitted
`LED_GREEN`	26	Green LED	OUTPUT	330Ω series resistor fitted
`LED_BLUE`	32	Blue LED	OUTPUT	Shared with I2S BCLK
`BTN_NEXT`	13	NEXT button	INPUT_PULLUP	LOW when pressed
`BTN_ENT`	14	ENT button	INPUT_PULLUP	LOW when pressed
`TFT_CS`	5	TFT chip select	OUTPUT	Boot sensitive
`TFT_DC`	17	TFT data/command	OUTPUT	
`TFT_BL`	12	TFT backlight PWM	OUTPUT/LEDC	Init LOW first
`TFT_MOSI`	23	SPI MOSI	OUTPUT	Shared: TFT/Touch/SD
`TFT_SCLK`	18	SPI clock	OUTPUT	Shared: TFT/Touch/SD
`TFT_MISO`	19	SPI MISO	INPUT	Shared: TFT/Touch/SD
`TOUCH_CS`	33	Touch chip select	OUTPUT	XPT2046
`SD_CS`	15	SD card chip select	OUTPUT	Boot sensitive, pull-up fitted
`I2C_SDA`	21	I2C data	I2C	AHT20, OLED, expansion
`I2C_SCL`	22	I2C clock	I2C	AHT20, OLED, expansion
`ONE_WIRE`	4	DS18B20 one-wire	INPUT/OUTPUT	4.7k pull-up fitted
`LDR`	39	LDR light sensor	ADC1 INPUT	Input only pin
`MIC_ANALOG`	36	Analog microphone	ADC1 INPUT	Input only pin
`MEMS_BCLK`	32	I2S bit clock	I2S OUTPUT	Shared with LED_BLUE
`MEMS_WS`	35	I2S word select	I2S OUTPUT	Input only at GPIO level
`MEMS_SD`	34	I2S data	I2S INPUT	Input only pin

DAC_AUDIO	25	DAC audio output	DAC OUTPUT	True analog out
PIEZO	27	Piezo buzzer	OUTPUT/LEDC	PWM tone via LEDC

Reserved — never use: GPIO 6, 7, 8, 9, 10, 11 are connected to internal flash SPI. Using these will crash the board immediately.

6 Practical exercise

This exercise brings together GPIO output, analog input, and LEDC PWM in one sketch. The TFT backlight brightness will track the ambient light level — brighter room, brighter display.

```
#include <Arduino.h>

// ── LEDC backlight setup ──────────────────────────────────
const int BL_CHANNEL = 0;
const int BL_FREQ    = 5000;
const int BL_RES     = 8;

// ── Timing ────────────────────────────────────────────────
unsigned long lastRead   = 0;
unsigned long lastReport = 0;
const unsigned long READ_INTERVAL   = 100;   // Read LDR 10x/sec
const unsigned long REPORT_INTERVAL = 2000;  // Report every 2s

// ── State ─────────────────────────────────────────────────
int currentBrightness = 255;
int ldrRaw = 0;

void setup() {
  Serial.begin(115200);
  Serial.printf("T02 Exercise — Auto Backlight  FW: %s\n", FW_VERSION);

  // LEDs
  pinMode(LED_RED,   OUTPUT);
  pinMode(LED_GREEN, OUTPUT);
  digitalWrite(LED_RED,   LOW);
  digitalWrite(LED_GREEN, HIGH);  // Green = running

  // Backlight — init LOW before LEDC
  pinMode(TFT_BL, OUTPUT);
  digitalWrite(TFT_BL, LOW);
  ledcSetup(BL_CHANNEL, BL_FREQ, BL_RES);
  ledcAttachPin(TFT_BL, BL_CHANNEL);
  ledcWrite(BL_CHANNEL, 255);

  Serial.println("Ready — cover the LDR to dim the backlight");
}

void loop() {
  unsigned long now = millis();

  // Read LDR and update backlight
  if (now - lastRead >= READ_INTERVAL) {
    lastRead = now;

    ldrRaw = analogRead(LDR);

    // Map LDR reading to backlight brightness
    // LDR reads HIGH in bright light, LOW in dark
```

```
    currentBrightness = map(ldrRaw, 0, 4095, 10, 255);
    currentBrightness = constrain(currentBrightness, 10, 255);

    ledcWrite(BL_CHANNEL, currentBrightness);
  }

  // Report to Serial every 2 seconds
  if (now - lastReport >= REPORT_INTERVAL) {
    lastReport = now;
    Serial.printf("[%lu ms] LDR: %4d  Brightness: %3d\n",
                  now, ldrRaw, currentBrightness);
  }
}
```

6.1 What to observe

- Cover the LDR with your finger — the backlight should dim smoothly
- Uncover it — backlight returns to full brightness
- The Serial monitor shows LDR readings and brightness values every 2 seconds
- The GREEN LED lights on startup confirming the firmware is running
- The backlight never goes fully off — constrain() keeps a minimum of 10

Try this

Change the map() minimum from 10 to 128. This sets a minimum brightness of 50% — useful for a real product where you don't want the display to go too dark. Notice how constrain() catches any out-of-range values that map() might produce at the edges.

7 T02 summary

Concept	Key takeaway
GPIO modes	OUTPUT for driving things. INPUT_PULLUP for buttons to GND. Never leave inputs floating.
Input-only pins	GPIO 34, 35, 36, 39 cannot drive outputs. Hardware peripherals (I2S, ADC) bypass this restriction.
Boot-sensitive pins	GPIO 0, 5, 12, 15 are sampled at boot. The E4 manages these correctly but initialise TFT_BL LOW before using LEDC.
build_flags -D	-DLED_RED=16 is identical to #define LED_RED 16 in every file. Global, automatic, no includes needed.
ADC1 rule	When WiFi is active, only GPIO 32-39 are safe for analog reads. ADC2 pins conflict with the WiFi radio.
LEDC PWM	ledcSetup(ch, freq, res) → ledcAttachPin(pin, ch) → ledcWrite(ch, 0-255). Init the pin LOW before calling ledcSetup.
map() and constrain()	map() scales between ranges. constrain() clips to min/max. Use both together for robust sensor-to-output conversions.

What's next

- T03 — Buttons and debouncing: the NEXT and ENT buttons are ready — now let's make them reliable with a proper debounce implementation that replaces the delay(200) approach from T01
- T04 — I2C fundamentals: with GPIO and ADC under your belt, reading your first I2C sensor — the AHT20 already on the E4 — is the natural next step